Slate – Hands Free Gesture Based Whiteboard

1. Introduction

1.1 Project Overview

Slate is a web-based collaborative whiteboard controlled entirely through hand gestures. Rather than using a mouse or pen, users draw, pan, zoom and select tools by moving their hands in front of a standard webcam. The application tracks hand landmarks using Google's MediaPipe library and translates gestures such as a pinch to draw or an open palm to stop, into canvas actions in real time.

The project is built with Next.js 16 (React 19, TypeScript) on the frontend, using Konva for canvas rendering and a Web Worker to run gesture recognition without blocking the UI. The backend is a custom Node.js server with Socket.IO for real-time collaboration, MongoDB for data storage using JSON and JWT authentication. The whole app is containerised with Docker Compose.

Key features include gesture-driven colour and brush size selection, undo/redo, whiteboard sharing with live notifications and screen recording.

1.2 Problem Statement

Existing digital whiteboards all rely on conventional input like mouse, trackpad or stylus. This presents several issues:

- **Accessibility** — users with motor impairments or repetitive strain injuries may struggle with these input methods.
- **Unnatural interaction** — drawing with a mouse feels disconnected from the natural act of sketching on a real whiteboard.
- **Hardware cost** — devices like SMART Boards or drawing tablets address some of these issues but are expensive and impractical for most individuals.

What is missing is a free browser-based whiteboard that works with just a webcam that nearly every laptop already has. Slate was built to fill this gap: a hands-free, gesture-controlled whiteboard that requires no additional equipment.

1.3 Objectives & Goals

Primary Objectives:

1. **Gesture-controlled drawing** — use webcam-based hand tracking as the sole input for drawing on the canvas.
2. **Real-time collaboration** — allow multiple users to work on the same whiteboard simultaneously via WebSockets.
3. **User authentication** — registration, login and JWT session management so whiteboards are linked to user accounts.
4. **Canvas persistence** — save whiteboard state to MongoDB so users can reload their whiteboards.
5. **Core whiteboard tools** — colour selection, brush sizing, undo/redo, panning and clearing.
6. **Containerised deployment** — package the app with Docker Compose for an easy and lightweight deployment.

Secondary Objectives:

7. **Whiteboard sharing** — share boards with other users with real-time Socket.IO notifications.
8. **Session recording** — screen recordings that can be uploaded, replayed and downloaded.

9. **Smooth tracking** — apply a smoothing algortihm to hand landmark data to reduce jitter.
10. **Performant processing** — run MediaPipe in a Web Worker to keep the main thread responsive.

1.4 Scope & Limitations

In scope: A fully functional, deployable web application covering the complete development lifecycle — requirements, design, implementation, testing, deployment, and this report.

Limitations:

- **Mobile** — designed and tested for desktop browsers only.
- **Offline mode** — an internet connection is required for authentication, persistence and collaboration.
- **Security hardening** — JWT auth and httpOnly cookies are implemented but no formal security audit, rate limiting or CSRF protection was carried out. The app is not fully ready yet for large scale production.
- **Browser compatibility** — tested primarily in Chrome and Safari. Other browsers were not formally verified.
- **Scalability** — tested with a small number of concurrent users and no load testing was performed.

2.Background Research [Darragh]

2.1 Existing Collaborative Whiteboard Tools

With the rise of remote and hybrid working structures, along with the ever-increasing public reliance on the Internet, the demand for a collaborative whiteboard tool has never been higher. Essential to beginning this project was understanding the landscape in which we are bringing it to birth, otherwise we risk positioning ourselves as a redundant imitator rather than a distinct tool.

Naturally, with this growing demand, several suppliers have stepped up to try and meet the need. Chief among these tools is Miro, a sort of pseudo-standard canvas tool that incorporates "sticky-note" widgets along with a host of other features to encourage free-form visual collaboration. Miro supports synchronous and asynchronous collaboration, boasts over 100 third-party integrations and offers flexible approaches to sharing and roles, with configurable access to allow for adaptive collaboration.

In contrast, the Microsoft Whiteboard tool benefits from foundational integration with the Microsoft 365 ecosystem, making it a natural choice for organisations committed to tools like Teams and OneDrive. It offers "sticky notes" and real-time co-authoring, similar to Miro, but whiteboards are saved to OneDrive where they can be revisited asynchronously. The platform provides basic functionality rather than advanced facilitation tools, making it more of a convenient built-in option than a specialist tool.

Across these tools, and the many others that become available year-on-year, we see certain patterns emerge. All provide infinite canvases with basic drawing tools. All offer collaboration with others, whether synchronously or asynchronously. Many now offer templates for common workflows and integrations with other prevalent tools.

However, the focus of this project is the gap we see across all of these tools; they all rely exclusively on conventional input methods. For the vast majority of users, this means a mouse or trackpad, which, while ubiquitous tools, provide little utility as implements for drawing. Drawing with a mouse feels disconnected from the natural gesture of drawing with pen or pencil, and similarly drawing with a trackpad feels unintuitive and awkward. While users could utilise a stylus with a tablet-like touchscreen, the requirement for external hardware is a significant barrier to entry for the basic function of these tools.

Therefore, what is missing is a whiteboard tool which, without need for external hardware, allows for natural and intuitive interaction. It is from this standpoint that we approach the concept of gesture recognition.

Though deep learning models for gesture recognition improve year-on-year, there are no mainstream collaborative whiteboard tools utilising this new technology, despite the ubiquity of laptop webcams. Using only that basic webcam of a laptop, we can use the deep learning MediaPipe framework provided by Google to track on-screen hands and recognise the motions made, which are then mapped to basic drawing functions. This is the niche that Slate aims to fill. A collaborative whitebaord where users draw, pan, and zoom through natural hand movements, requiring no costly additional hardware. By combining this nascent technology with capabilities akin to existing products, we open whiteboarding to new use cases.

2.2 Real-Time Communication Technologies (WebSockets, Socket.IO)

2.3 Gesture Recognition in Web Applications

---

3. Requirements Analysis

3.1 Functional Requirements

3.2 Non-Functional Requirements

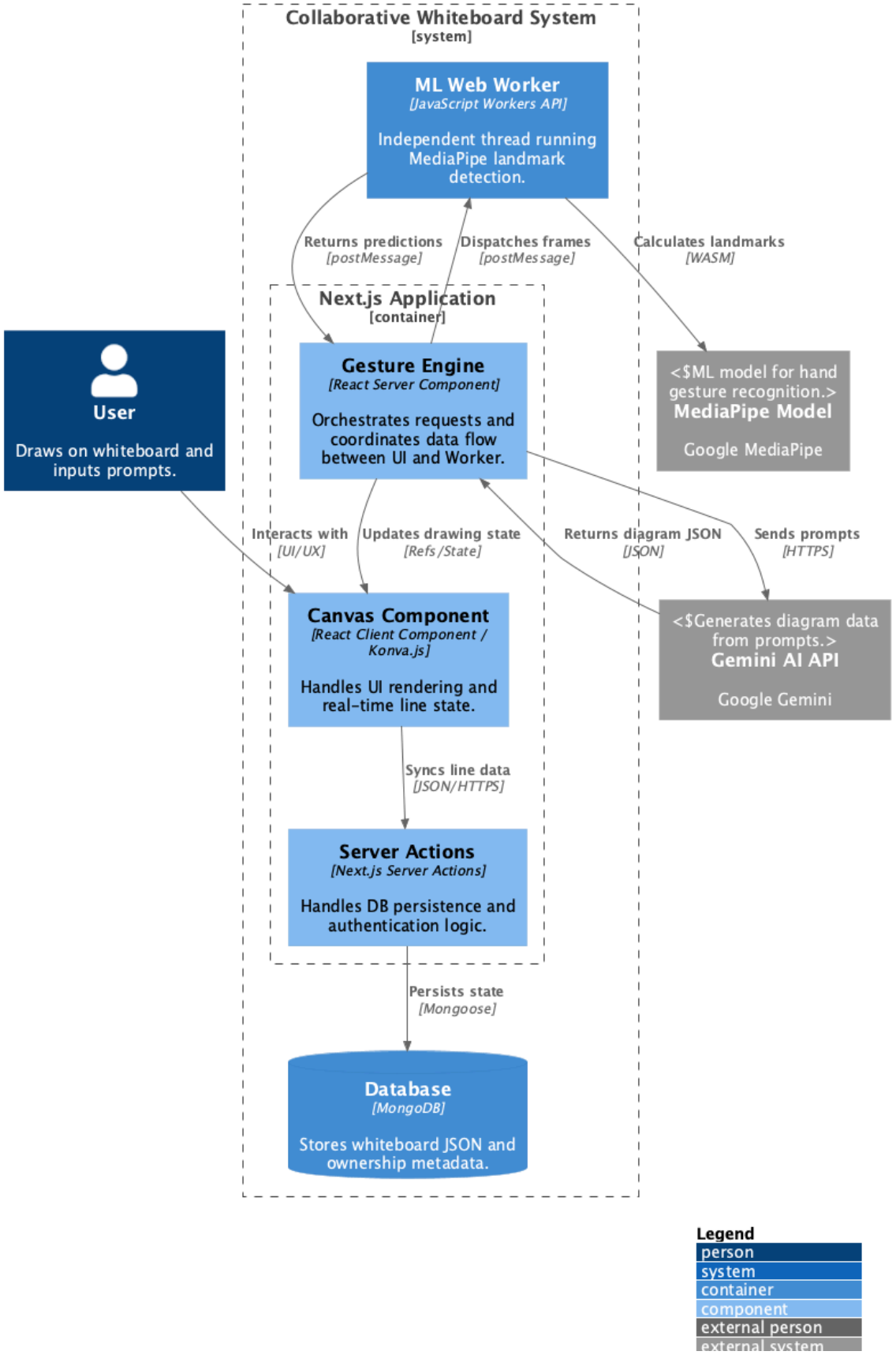
3.3 User Stories / Use Cases

---

4. System Architecture & Design [Naoise]

4.1 High-Level C4 Architecture Diagram

Container Diagram for Collaborative AI Whiteboard



4.2 Tech Stack

4.2.1 Frontend

4.2.2 Backend

4.2.3 Deployment

4.3 Database Schema Design

4.4 API Design

4.5 Real-Time Communication Design

---

5. Implementation

5.1 Authentication System

The authentication system handles registration, login and session management. It was one of the first features built because almost everything else in the app depends on knowing who the current user is. The system is built with Next.js Server Actions, JWT tokens and httpOnly cookies.

5.1.1 Registration & Login Flow

Both the login and register pages share a reusable `AuthForm` component that renders a username/password form. The pages are thin client components that wire the form up to a Server Action using React 19's `useActionState` hook. This gives us a clean way to call a server-side function from a form submission and get back validation errors without writing manual fetch calls or API routes.

When a user submits the register form the `registerAction` in `src/app/(auth)/actions/register.ts` runs on the server. It first validates the input with a Zod schema defined in `src/lib/validation/auth.ts` which enforces that usernames are 3 to 20 characters containing only letters, numbers and underscores and that passwords are at least 8 characters. If validation fails the action returns human readable error messages in a red banner.

If validation passes the action connects to MongoDB, checks whether the username is taken and if not hashes the password with `bcryptjs` using a salt of 10 rounds before creating the user document. The User model in `src/app/models/User.ts` is a straightforward Mongoose schema with `username`, `password` and `createdAt` fields. After creating the user it generates a JWT containing the `userId` and `username` and sets it as an httpOnly cookie named `session` with a 7 day expiry. Once set the action calls `redirect("/")` to send the user to the home page.

The login flow in `loginAction` follows the same structure. It validates with the same Zod schema, looks up the user by username and uses `bcrypt.compare` to check the password against the stored hash. It deliberately returns the same generic "Invalid username or password" message whether the username does not exist or the password is wrong to avoid leaking which usernames are registered. On success it creates a JWT and sets the cookie just like registration does. Logout simply deletes the session cookie since JWTs are stateless and no server-side invalidation is needed.

One thing worth noting: the `redirect()` call in both actions is placed **outside** the try/catch block. Next.js internally throws an error to perform a redirect so if that throw gets caught the redirect silently fails. This

tripped us up during development until we figured out what was happening.

5.1.2 Session Management & Middleware

Session management revolves around the JWT stored in the `session` cookie. The `src/lib/auth.ts` file contains all the token utilities. `createToken` builds a JWT signed with HS256 using `jose` with a 7 day expiration. `verifyToken` decodes a token and returns null if it is expired or tampered with. `getSession` reads the cookie and calls `verifyToken`. `getAuthenticatedUser` goes further by looking up the full user document from MongoDB using the `userId` in the payload which is needed in places like the whiteboard page.

Route protection is handled by Next.js middleware in `src/middleware.ts`. It runs on every request matching the `matcher` pattern covering `/login`, `/whiteboard`, `/home`, `/settings` and `/recordings`. The middleware reads the `session` cookie and verifies it with `jwtVerify`. If someone tries to access a protected route without a valid session they get redirected to `/login`. If a logged in user visits `/login` they get sent to `/whiteboard` since there is no reason for them to see the login page again.

Using middleware rather than checking the session inside each page gives us a single source of truth for access control that runs before any page rendering. It also keeps the page components clean since they do not need to handle auth checks themselves.

5.2 Whiteboard Canvas

5.2.1 Drawing Engine & Canvas API

5.2.2 Colour Selection & Size Selector

5.2.3 Undo/Redo System

5.2.4 View Transform (Pan/Zoom)

5.3 Real-Time Collaboration

5.3.1 Socket.IO Integration

5.3.2 Canvas State Synchronization

5.3.3 Shared Canvas & Permissions

5.4 Gesture Recognition

5.4.1 GestureEngine & Custom Gestures

5.4.2 Web Worker for Performance

5.4.3 1€ Filter for Smoothing

5.5 Recording Feature

5.5.1 Recording Controls

5.5.2 Upload & Storage

5.6 Camera Integration

---

6. Deployment

6.1 Docker Containerization

We used Docker to containerise Slate so the application builds and runs the same way regardless of the host machine. This avoided issues like having to manually install all dependencies and packages.

The Dockerfile uses a multi-stage build with three stages:

1. **deps** - Installs npm dependencies on a `node:20-alpine` base image using `npm ci --legacy-peer-deps`. The flag was needed because some dependencies (React 19 and framer-motion) had conflicting peer requirements.

2. **builder** - Copies `node_modules` from the first stage along with the source code and runs `npm run build` to produce the Next.js production build.

3. **runner** - The final image. Only copies what is needed to run: `node_modules`, `.next`, `public`, `server.mjs`, `package.json` and `next.config.ts`. It exposes port 3000 and starts the app with `node server.mjs`.

This multi-stage approach keeps the final image small by discarding source code and build tooling.

6.2 Docker Compose Configuration

Docker Compose wraps this into a single command. The `docker-compose.yml` defines one service:

```yaml
services:
  slate:
    build:
      context: ./slate
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    env_file:
      - ./slate/.env.local
    restart: unless-stopped
```

- **ports** maps container port 3000 to the host so the app is available at `http://localhost:3000`.
- **env_file** loads secrets from `.env.local` which is not committed to the repo.
- **restart: unless-stopped** automatically restarts the container oif it crashes.

To deploy the application:

```
docker compose up --build
```

6.3 Environment Configuration

Slate uses a `.env.local` file in the `slate/` directory for environment specific config. The required variables are:

`MONGODB_URI` :Connection string for the MongoDB Atlas cluster.

`JWT_SECRET` :Secret key used to sign and verify JWT session tokens

MongoDB Atlas handles database hosting externally so the Docker setup only needs to run the Next.js app itself. If either variable is missing the app falls back to hardcoded defaults which is fine for local development but would need to be changed for a real deployment.

---

7. Testing

7.1 Testing Strategy

7.2 Unit Tests

7.3 Integration Tests

7.4 User Acceptance Testing

7.5 Test Results & Coverage

---

8. Project Management (Oisín)

8.1 Team Roles & Responsibilities

8.2 Methodology (Agile/Scrum)

The team chose to use the Agile development methodology with elements of the Scrum framework blended into it. The Agile methodology is just that, a methodology whereas Scrum is a framework for Agile project management that can be followed. It is best to use the methods that allow the project to succeed rather then be held back in trying to follow a framework to the letter.

Agile methodology suited the collaborative nature of the environment in which the project was developed. The size of the group and the list objectives to be achieved would have been significantly hampered by Waterfall with team members waiting for other pieces of the process to be completed before they could even begin working. The Waterfall methodology would have removed the collaborative environment in meetings which allowed new ideas to be pitched at all times to solve individual parts of the design. This open and discussion heavy space needed the Agile methodology to ensure the group was adaptable in a fast moving development environment.

Parts of Scrum that were utilised were the Sprints, Sprint planning, Sprint demo and Project Lead. The Sprints gave structure to an open project development environment by providing immediate deadlines week on week to ensure that new integrations, bug fixes and changes were continuously made. This was in an attempt to have continuous improvement to the deployable product so as to meet the aforementioned short deadline. Sprint Planning took place each Wednesday before or after the meeting with Klaas as did the Sprint demo. Instead of separating these into separate events they were combined into one each week as the group as a majority, not always a whole, were present. Team members could display there changes to those present and the features function, viability and necessity would be evaluated as a group there. The goal each week was to deliver what was planned in the Sprint Planning the week prior. The product backlog would be looked at to find wanted features and based on whom previous worked in an area of the project the PBI would be given to the team member for which it made most sense to complete it. Given there was no customer to give feedback on the weekly sprint, the team became the Scrum Product Owner constantly evaluating and analysing everything committed to the code base to ensure it was pushing the project in the correct direction.

This Agile environment allowed the group to work fast and in parallel, enabled by Git, to keep development moving quickly and iteratively. The iterative nature of the process and frequent communication benefited the

project as it allowed the team to be adaptable and make changes rapidly and when needed. The use of these methodologies and frameworks was a significant contributor to the success of the project.

8.3 Sprint Breakdown & Timeline

As stated above we used Agile and Scrum to complete this project which consisted of **one week Sprints** allowing for continuous integration and improvement and the flexibility to modify requirements.

Sprints

1. - Inception of idea and the creation of the basic JavaScript file to test viability.
   - Apply smoothing algorithm to drawing gestures.
   - Select gestures to be used for drawing.
2. - Integration of the basic "Whiteboard" with React.
   - Improvement in smoothing algorithm, Gesture Engine implementation.
   - Creation and integration of screen recording to whiteboard.
   - Refactor and removal of redundant code.
3. - Creation and setup of MongoDB database for storing saved 'Slates', recordings and user details.
   - Addition of JWT token session management.
   - Implementation of Register and Login features.
4. - UI of landing page made.
   - Homogenisation of the UI for a consistent feel.
   - Consolidated routing to pages within website.
   - Introduction of gesture based panning and zooming.
   - Consolidated saving to MongoDB Atlas Cluster
5. - Create Saved whiteboard function and optimise.
   - Creation and optimisation of file sharing.
   - Colour spinner created and integrated.
   - Settings menu created and designed with CSS.
   - Addition of Copy/Paste feature
6. - Improvement panning, colour wheel and sidebar.
   - Integration of Undo/Redo through gestures.
   - Integration of keybinds for undo,redo,cut.
   - Improvement to settings page with a keyboard shortcut for access and display improvements.
   - Add colour white to colour wheel.
7. - Addition of gesture based zoom.
   - Moved undo/redo gestures to left hand.
   - Rectified issues with screen recording.
   - Addition of promptable LLM that can draw for you.
   - Ensure main branch is functioning minimum viable product for presentation.
8. - Setup Docker files for containerisation
   - Finalise products features.
   - Create Docker image and deploy.

A deployable version of the product was ready by week 8 for a live demonstration during the presentation and a fully fledged version was completed by week 9 of the semester. The goal was to have the fully fledged product available for the presentation but due to the requirement for it to be functioning for the presentation certain features had to be compromised.

8.4 Version Control Strategy (Git)

Our version control strategy was implemented using Git which is an industry standard for version control in software development environments. Given that version control is the backbone for any successful software project it was important clear guidelines were laid out and a system was followed.

The group settled on Feature Branching as the branching and development strategy for the project. This was selected over GitFlow and Trunk-based development as it suited the Agile nature of the team. Trunk-based combined with our 7-person team would have become far too messy as this requires 'main' to be the only branch and all changes to be made and committed here. GitFlow overcomplicated how development would occur as it needed multiple branches such as 'main', 'develop' and 'release' on top of all the individual 'feature' branches. Feature Branching however keeps 'main' as the deployable branch into which each feature from feature branches are integrated. Feature Branches are isolated versions of 'main' where each new feature is developed whilst avoiding introducing bugs etc. into the deployable 'main' branch. This isolation facilitated the Agile nature of the groups workflow allowing separate and contrasting features to be developed in parallel. Before pushing the new feature it was made clear that 'main' must be pulled into the branch to ensure the feature worked correctly in harmony with any changes integrated during your individual development time and that it didn't introduce any bugs into working code.

Commits containing a whole purpose however frequent were to be made rather combining multiple purposes into one large commit. This can compromise the codebase as if a bug was to be introduced in a commit with multiple changes the difficulty of finding and fixing this bug is far greater than if the commit contained a single unit of development which could be examined for mistakes. Commit messages were to be clear, concise and relevant to what was being committed, this ensured a clean git log and commit history. Tags were not used in this project due to the short time span in which it was developed the only 'version' created was the final presented one.

If a commit was reliant or impacting upon another feature group members were required to sufficiently communicate their progress to each other before committing to prevent either party having a negative effect on the code base or each others features. Changes were to be pushed as frequently as necessary to make them available for other team members to interact with on their own branches as mentioned previously.

By adhering to our own guidelines it kept consistency amongst the group and allowed the project to be delivered on time with minimum errors and mishaps.

8.5 Communication & Collaboration Tools

The team communicated on a daily basis through a WhatsApp group chat created at beginning of this process, this allowed for quick updates, information sharing and collaboration between team members outside of the university campus. This greatly increased efficiency of the development process providing instant access to resources and information. The existence of WhatsApp for the web made it the only communication channel used as this allowed for links to sites and other resources to be accessed on one's own laptop or PC rather than their mobile phone.

The primary collaboration tool used was Git as detailed above facilitating the development and integration of differing features into one deployable final product. For the presentation a shareable PowerPoint was made to allow each speaker to make their own individual slides, this kept the styling consistent throughout the slide deck.

---

9. Evaluation & Discussion (Rakib)

9.1 Objectives Met vs. Not Met

9.2 Technical Challenges & Solutions

9.3 Performance Analysis

---

10. Future Work

10.1 Planned Enhancements

10.2 Scalability Considerations

10.3 Additional Features

---

11. Conclusion

---

12. References

---

13. Appendices

Appendix A: Full API Documentation

Appendix B: Database Schema Details

Appendix C: User Manual / Screenshots

Appendix D: Meeting Minutes / Sprint Logs

Appendix E: Individual Contributions